

C.6.1 Proof of Lemma C.26

We first reduce another problem in communication complexity to IsTree.

Lemma C.28. *Consider the following two-party game, where Alice receives string $x \in \{0, 1\}^n$ and Bob receives an integer k , and Bob wants to know whether $x_k = x_{k-1}$. If only Alice is allowed to send information, $\Omega(n)$ bit communication complexity is required.*

Proof of Lemma C.28. We can reduce this problem to the problem in Lemma C.24. Considering the game in Lemma C.24, given any $x \in \{0, 1\}^n$, we can construct $\tilde{x}_i = \sum_{j=1}^i x_j \bmod 2$. Then $\tilde{x}$ is a string of length n with only 0 and 1. Moreover, $x_k = x_{k-1}$ if and only if $\tilde{x}_k = \tilde{x}_{k-1}$. Hence, if Bob can solve the problem in Lemma C.28 with $o(n)$ bit, he can solve the problem in Lemma C.24. This is a contradiction. $\square$

Proof of Lemma C.26. Now suppose that we have a streaming algorithm for IsTree with only $o(n)$ memory. We shall prove Alice and Bob in Lemma C.28 can use it to solve the original question with $o(n)$ memory.

Consider the following graph with $n + 2$ nodes. There is a node i corresponding to each x_i for $i \in [n]$ and two special nodes $n + 1, n + 2$. Node i will be connected to $n + 1$ if $x_i = 0$ and to $n + 2$ if $x_i = 1$. Moreover, $k - 1$ and k will be connected. Now the original answer Bob wants is False if and only if the graph is a Tree. Hence, given access to the streaming algorithm, Alice can run it on the edges that she knows exist and send the memory to Bob. Bob can then run it on the edges that he knows exist. Combining they will be able to solve the original problem. This is a contradiction.

Moreover, as RNN with CoT is also a streaming algorithm, it also requires $\Omega(n)$ memory. $\square$

C.6.2 Proof of Lemma C.27

Proof of Lemma C.27. The proof is two-folded. We will first define an algorithm that can solve IsTree by generating a sequence of vertices of length $O(n)$, and then we will show that this sequence can be generated by a Transformer with constant depth and width, and $O(\log n)$ precision as a Chain of Thought.

Algorithm 1 Depth-First Search Algorithm

Require: A graph $G = (V, E)$ with n vertices and E has an ordering $e_1, \dots, e_m$.

- 1: Initialize two stacks of vertices S_1, S_2 with $S_1 = [v_1], S_2 = \emptyset$.
 - 2: **while** S_1 is not empty **do**
 - 3: Let v be the top of S_1 . Yield v .
 - 4: **if** there exists a neighbor u of v not in $S_1 \cup S_2$ **then**
 - 5: Choose u such that edge (u, v) has the smallest possible order and push u to S_1 .
 - 6: **else**
 - 7: Pop v from S_1 and push v to S_2 .
 - 8: **end if**
 - 9: **end while**
-

Algorithm for IsTree. We define Algorithm 1 as a depth-first search algorithm that can generate a sequence of vertices of length $O(n)$ that can be used to solve IsTree. We will use two stacks S_1, S_2 to store the vertices. S_1 will be used to store the vertices that are not yet visited, and S_2 will be used to store the vertices that are already visited. The algorithm will start with $S_1 = [v_1]$ and $S_2 = \emptyset$. At each step, the algorithm will pop the top of S_1 and push it to S_2 . Then it will push all the neighbors of the popped vertex that are not in $S_1 \cup S_2$ to S_1 . The algorithm will terminate when S_1 is empty. We will denote the yielded vertex sequence for G as $\mathcal{A}(G)$. The following lemma shows the connection between the result of the algorithm and the IsTree problem.

Lemma C.29. *For any graph G , $\mathcal{A}(G)$ is a tree traversal of a spanning tree of the connected component of G containing v_1 . Hence IsTree(G) is True if and only if G has $n - 1$ edges and $\mathcal{A}(G)$ contains $2n - 1$ vertices.*

Proof of Lemma C.29. First, every vertex in the connected component of G containing v_1 will be visited. This is because the algorithm will always push all the neighbors of the popped vertex that are not in $S_1 \cup S_2$ to S_1 . Hence, the algorithm will terminate when all the vertices in the connected component of G containing v_1 are visited.

Second, every two consecutive vertices in the yielded sequence will be connected by an edge. This is because the algorithm will always push one of the neighbors of the popped vertex that is not in $S_1 \cup S_2$ to S_1 . Hence, every two consecutive vertices in the yielded sequence will be connected by an edge. On the other hand, the combination of

these edges will form a tree because the algorithm will never push a vertex that is already in $S_1 \cup S_2$ to S_1 . Hence, the yielded sequence is a tree traversal of a spanning tree of the connected component of G containing v_1 . $\square$

Construction of Transformer. We will now show that the yielded sequence of Algorithm 1 can be generated by a Transformer with constant depth and width, and $O(\log n)$ precision as a Chain of Thought. The Transformer will generate a valid yielded sequence but can terminate early if the graph is not a tree. We will now describe the Transformer in detail. We will assume the input token sequence S is as follows,

$$S = \text{Tokenize}(G), v_1, \dots, v_r \quad (10)$$

for some $r \geq 0$ and $v_1 \dots v_r$ is a valid yielded sequence. The length of $\text{Tokenize}(G)$ is $3n - 2$ with 3 tokens for each edges and 1 special token $\langle s \rangle$. We will further denote the input to the first layer X as $\text{Emb}(S)$. We will similarly denote the input to layer ℓ as $X^{(\ell)}$. We will also denote the output of the last layer as X^{out} .

1. **Layer 1 and Layer 2 Attention.** The attention at Layer 1 will output zero and the FFN at Layer 1 and Attention at Layer 2 will implement a counting function (Definition C.8) to count the number of vertices n appears in the previous token sequence and write $\text{ROUND}(\frac{1}{n}, p)$ in a new dimension i_1 as a result.
2. **Layer 2 FFN and Layer 3 Attention.** The FFN at Layer 2 and Attention at Layer 3 will implement a copying function (Definition C.7) copying the first dimension and the counting dimension i_1 of each token to its successor at two new dimensions i_2 and i_3 . For each edge, this moves the type of the first vertice and the number of times the first vertice appears to $\sim$. For every vertice in the chain of thought, this moves the type of the previous vertice to them.
3. **Layer 3 FFN and Layer 4 Attention.** The FFN at Layer 3 and Attention at Layer 4 will implement another copying function, copying the dimensions i_2 and i_3 of each token to its successor at two new dimensions i_4 and i_5 . Especially, for each edge, this moves the type of the first vertice and the number of times the first vertice appears to the position corresponding to the second vertices.
4. **Layer 4 FFN.** This FFN will process the information gathered from the previous layer and prepare for the next layer. It will make sure the following properties hold for $X^{(5)}$,
 - For every token, the position number, its square, and 1 will be kept in the last three dimensions.
 - For the first vertices in each edges, $\sim$ and $\langle s \rangle$ The rest dimension will be zero.
 - For the second vertices of each edges (a, b) , there will be four dimensions i_6, i_7, i_8, i_9 with value a, b and $n_{a,e}, n_{b,e}$, where $n_{a,e} = \text{ROUND}(\frac{1}{1 + \#a \text{ appears up to current edge}}, 1)$.
 - For vertice v_l in $v_1, \dots, v_r$, there will be four dimensions $i_{10}, i_{11}, i_{12}, i_{13}$ with value v_l, v_{l-1} and v_l^2, v_{l-1}^2 ($v_0 = 0$).
5. **Layer 5 Attention.** Combining with the previous Layer 4 FFN layer, we will implement two match functions with two attention heads matching (i_{10}, i_{11}) or (i_{11}, i_{10}) with (i_6, i_7) at Layer 5 Attention, i.e. finding the edge in input for each step in the chain of thought, we will then copy $n_{v_l, (v_l, v_{l-1})}$ to dimensions i_8 and i_9 .
6. **Layer 6.** We will use Layer 5 FFN and Layer 6 Attention to implement the match function that matches dimension i_{10} of the current token to i_{10} in the previous token. This will match v_l to the first appearance of v_l in the chain of thought and we will copy i_{11} of the matched token to i_{22} . This dimension will be the first predecessor of v_l in the chain of thought (0 for v_1). We will denote this predecessor of v_l to be $f(v_l)$ as it is the father of v_l in the tree. Now we will need to split into two cases depending on whether v_{l-1} is $f(v_l)$. If $v_{l-1} = f(v_l)$ or $v_{l-1} = 0$ (for v_1), we will set dimension i_8 to be 1 and i_9 to be 0. Otherwise, we will keep dimension i_8 and i_9 as $n_{v_l, (v_l, v_{l-1})}$.
7. **Layer 7.** Now we will use Layer 6 FFN and Layer 7 Attention with two attention heads to implement two MatchNext functions⁵ (Definition C.12) which use i_8 or i_9 as the counting index, and match v_l at i_{10} to i_6 or i_7 respectively. We will then copy dimensions i_6 to i_9 of the matched tokens to i_{14} to i_{21} (because there will be two of them).

The match next function will be able to retrieve the first edge containing v_1 . For any $i \geq 2$, one of the matches next function will be able to retrieve the next edge containing v_i after (v_i, v_{i+1}) if it exists. If it doesn't exist, the corresponding counting dimension will either be zero or no smaller than $n_{v_i, (v_i, v_{i-1})}$. We will use Layer 6 FFN to decide whether the next edge exists and set dimension i_{14} of the output of Layer 6 to be the other edge in the next edge if it exists, or 0 otherwise, and i_{15}, i_{16} of the output of layer 6 to be the counting dimension of the next edge if it exists, or 0 otherwise. For each edge in the original input, we will also set dimension i_{15}, i_{16} to be the counting dimension of the edge.

⁵The constant A here in Definition C.12 is 1

8. **Layer 8 Attention** We will grab the next edge again, in the same manner as Layer 6, but this time using dimension i_{15} and i_{16} . The necessity of this step is that the next edge containing (v_{i-1}, v_i) in the original graph can be the same as the $(f(v_l), v_i)$ and in such case we need to check whether the next edge after this edge.
9. **Layer 8 FFN.** We now have, at each position corresponding to v_l , the first edge $(f(v_l), v_l)$ in the yielded sequence containing v_l and the other vertex in the edge containing v_l that hasn't been visited if it exists. If they don't exist, the corresponding dimension will be zero. This allows us to use Layer 8s FFN to decide the next vertex in the yielded sequence, which is exactly the first vertex different with $f(v_l)$ in the two edges if they exist, or $f(v_l)$ otherwise. We will use Layer 8 FFN to calculate the potential next vertex and put it in dimension i_{23} and its square in i_{24} .
10. **Layer 9 Attention.** Combining with Layer 8 FFN, we will match i_{23} of the current token to i_{10} of the previous token to find the first appearance of the potential next vertex in the chain of thought. We will then copy dimension d of the matched token to i_{25} . This value being 1 will imply this vertex has never appeared in the chain of thought before and any other value will imply the vertex has appeared before.
11. **Layer 9 FFN.** We can now check several cases,
 - If the potential next vertex v is either $f(v_r) \neq 0$ or never appears in the chain-of-thought sequence, then Layer 9 will output $n[-v, 1, -1, \dots, -1]$, which will decodes to v .
 - If the potential next vertex v is not $f(v_r)$ and appears in the chain-of-thought sequence, then Layer 9 will output nw_6 , which will decodes to NO, because the chain of thought has already visited v and hence the graph is not a tree.
 - If $v_r = 1$ and the potential next vertex v is $f(v_r) = 0$, this means the chain of thought has finished. In this case, layer 9 FFN will check whether the position is $3n - 2 + 2n - 1 = 5n - 3$ and output nw_5 if it is, or output nw_6 otherwise, which will decode to YES and NO respectively.

This concludes the construction of the Transformer. □

C.7 Proof of Theorem 4.8

We will now prove Theorem 4.8. We will first restate the theorem for convenience.

Theorem 4.8. *Given input length n , let R is an RNN with word embedding $W^{(E)} \in \mathbb{R}_p^{(n+n_S) \times d}$, where $p = \Theta(\log n)$ is the precision, the constant n_S is the number of special symbols in the vocabulary, the constant d is the embedding dimension. If each recurrent iteration can be computed by a circuit of size $C(n) \leq 2^{p/2}$, and if the RNN produces the final answer after running at most n^A steps of CoT for some constant $A > 0$, then there exist Transformers with $O(\log n)$ -bit precision, $O(C(n))$ parameters and word embedding $[W^{(E)} \quad \mathbf{0}^{(n+n_S) \times d}]$ that can produce the same final answer after running $(C(n) + 1)n^A$ steps of CoT.*

Proof. The proof is inspired by Theorem C.30 from Li et al. (2024).

Theorem C.30 (Theorem 3.3 of Li et al. (2024)). *For any n and any circuit with size $T(n)$ and input size n , there exists a Transformer with constant depth and precision, $O(\log n)$ width, and a position embedding with size $O(T(n) \log n)$, such that for any input S of length n , the Transformer computes the output of the circuit on S using $T(n)$ steps.*

However, direct utilization of Theorem C.30 is not feasible because we are interested in (1) $O(\log n)$ precision Transformer, and (2) simulating the RNN for n^A step, which would correspond to a circuit with $n^A C(n)$ in size. However, as the calculation is recurrent, we can encode the RNN circuit in $O(C(n))$ parameter instead.

To do so, we will unroll the circuit of each recurrent step of the RNN into $C(n)$ gates. We will then assign each gate a unique id in $[C(n)]$ and assume the circuit is calculated in the order of the gate id in the following manner.

1. Each gate has a type $t(i)$, which is either a constant gate outputting 1, an input gate, a hidden state gate, an AND gate, or an XOR gate.
2. Each gate i 's output depends on two values $l(i)$ and $r(i)$. If $t(i)$ is a constant gate, then $l(i)$ and $r(i)$ are assigned to be 0. When it is an input gate, $l(i)$ will be assigned to be the coordinate of the input embedding and $r(i)$ will be assigned to be the index of the bit of the value at $l(i)$ coordinate. When it is a hidden state gate, $l(i)$ will be assigned to be the coordinate of the hidden state embedding, and $r(i)$ will be assigned to be the index of the bit of the value at $l(i)$ coordinate. If it is an AND gate or an XOR gate, $l(i)$ and $r(i)$ will be assigned to be the id of the two gates that it depends on.